# Smart-RUDP

Reliable UDP in C, with a neural network that learns congestion control

Computer Networks | Faculty: Anuradha G | VIT Vellore

Harshavardhan Mallela 24BAI0274 Rohit Arya 24BCE0819 Shloak Sinha 24BDS0378

WHY

# UDP is fast but unreliable; TCP is reliable but locked in the kernel

### UDP

Fast and simple. Packets can be lost, duplicated or reordered, and nothing tells the sender.

### TCP

Reliable, but it lives inside the operating system: hard to see, hard to change, hard to learn from.

Our question

## Can we rebuild the core ideas of TCP ourselves on top of UDP, in a form we can read, tune and teach a neural network to control?

QUIC and HTTP/3 take the same route: reliability built in user space over UDP.

WHAT WE BUILT

# A complete system: protocol, learned controller, network and tools

### RUDP protocol

Handshake, Selective Repeat, CRC-16, adaptive timer. About 470 lines of C in the core.

### Smart-RUDP controller

A 5-32-32-5 neural network picks the window; safety net hands over to AIMD when loss is high.

### Network design

Four routers, OSPF, VLANs and an ACL in Cisco Packet Tracer, with failover.

### Applications and tools

Terminal and web chat, RUDP Drop file race, Wireshark dissector, a C library.

A student on VLAN 10 uploads a file to the server LAN. The ACL blocks staff VLAN 20. If a core link fails, OSPF reroutes through R2 and the transport adapts to the longer path.

RESULTS

# The neural agent beats tuned AIMD on every kind of path we tested

**15% to 52%**

less time than tuned AIMD on 4 of 5 paths (real clock, 10 paired runs)

**65% to 83%**

less time on the real Linux kernel (netem), significant on three paths

**0.53x**

the time of tuned AIMD across 100 random held-out paths; faster on 89

**4 of 4**

paths that change mid-transfer: neural agent fastest in every case

**0.84 vs 0.64**

Jain fairness with unequal RTTs, neural against AIMD

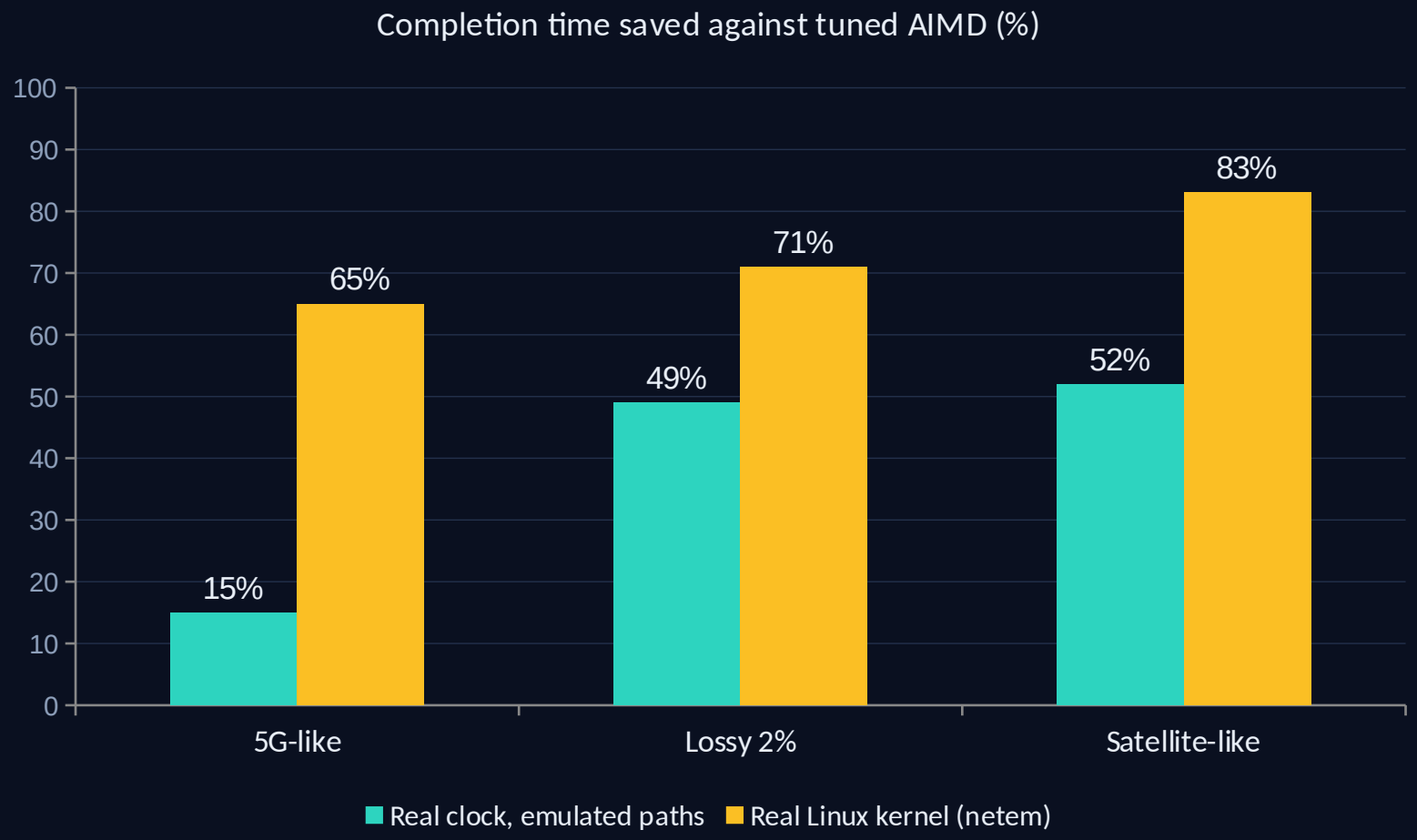
**24% to 34%**

link share next to a kernel TCP flow: it does not starve TCP

The same trained network is used in every test, with no retuning. Opponents are tuned AIMD and tuned CUBIC, not weak baselines.

RESULTS

# Up to 83% less time than tuned AIMD




- Same trained network on every path
- Real clock: 10 paired runs per path, 95% intervals
- Kernel: 3 to 5 runs per cell, all three significant
- Also 52% on the fast path, and 0.53x on 100 random paths

RESULTS

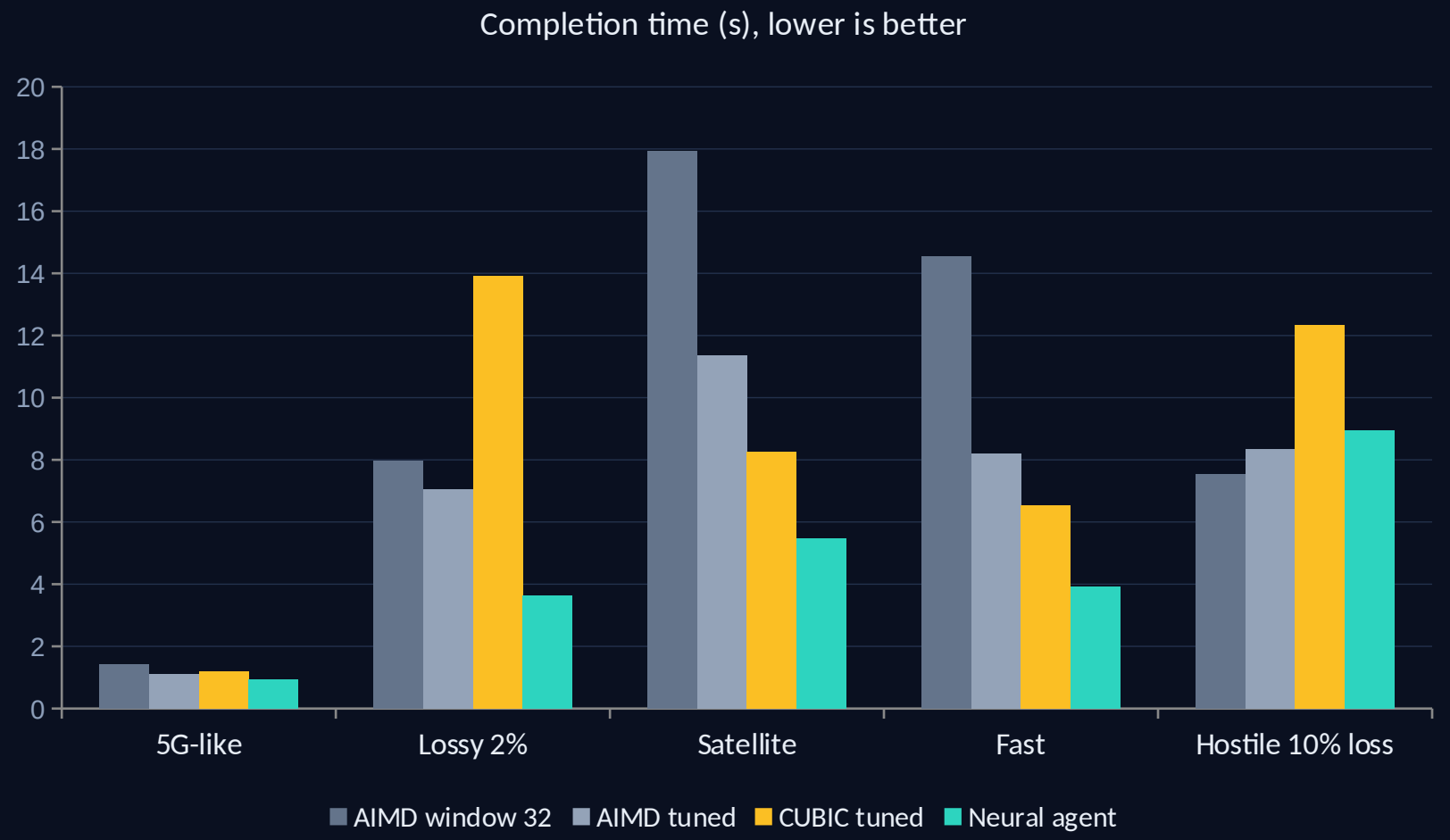
# Compared with every alternative we built or measured

| Alternative | Where measured | Neural agent result |
|---|---|---|
| Tuned AIMD | Real clock; real kernel | 15% to 52% less time (real clock); 65% to 83% (kernel) |
| AIMD, window 32 | Real clock; 2 MB file | Faster on every path; 2 MB lossy file 11.9 s vs 26.6 s |
| CUBIC (tuned) | Real clock, 10 paired runs | Faster on lossy (-74%) and satellite (-34%); equal on 5G and fast |
| BBR-style | Virtual time, 20 seeds | Wins satellite and fast; level on 5G; BBR-style wins lossy |
| Q-table agent | C transport, 15 runs | Table agent slower than AIMD on 5G and satellite; neural wins all three |
| Plain UDP | 48 simulated paths | Plain UDP delivered 68% of packets; RUDP delivers all |
| Kernel TCP (what FTP and SCP run on) | Real Linux kernel, netem | TCP is 1.5x to 1.7x faster than the neural agent, which is 3x to 6x faster than RUDP with AIMD |

FTP and SCP run on kernel TCP, so the kernel TCP row is the comparison; we did not run an FTP client itself.

RESULTS

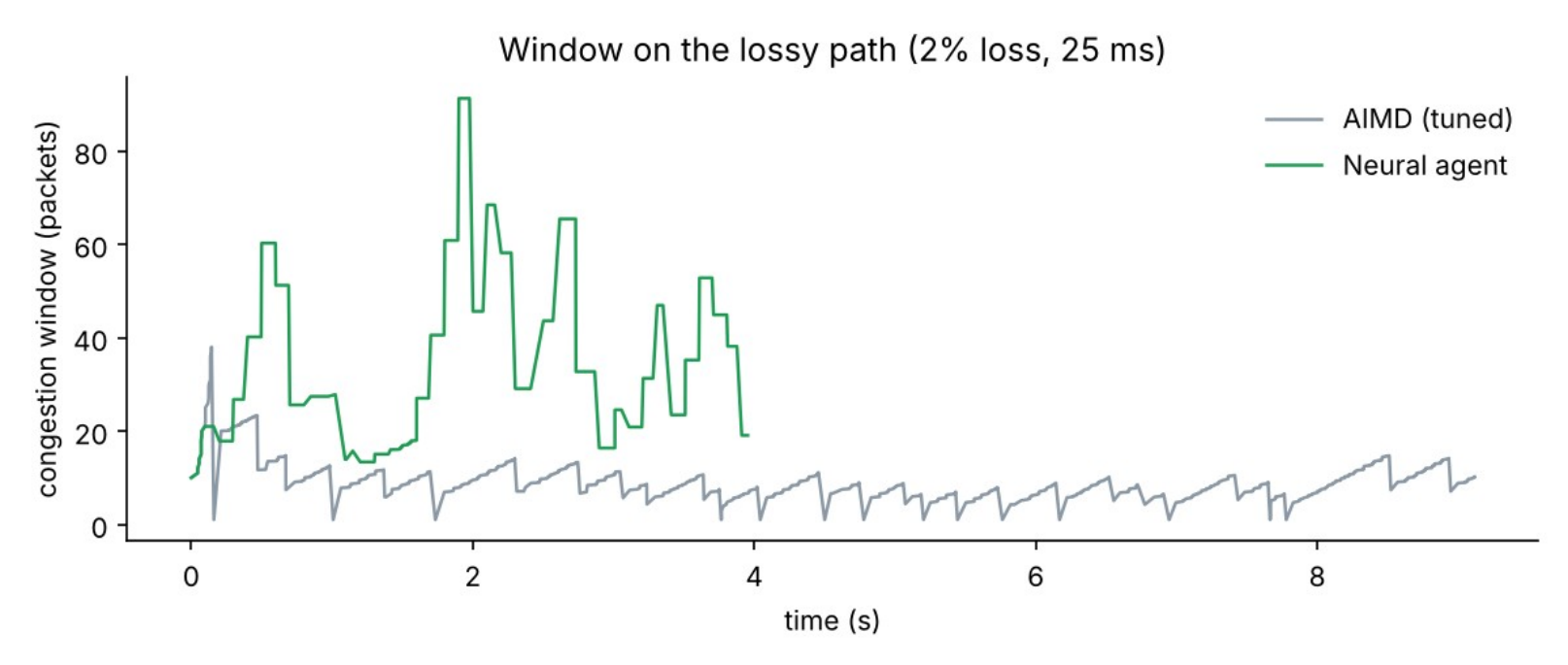
# Real clock: fastest on four of five paths against tuned baselines




- Neural agent fastest on 4 of 5 paths, significantly faster than tuned AIMD on all four
- Faster than tuned CUBIC on lossy (-74%) and satellite (-34%)
- Hostile 10% loss: no significant difference between any two policies
- Real clock, real server process; 10 paired runs per path (hostile: 5), 95% intervals

RESULTS

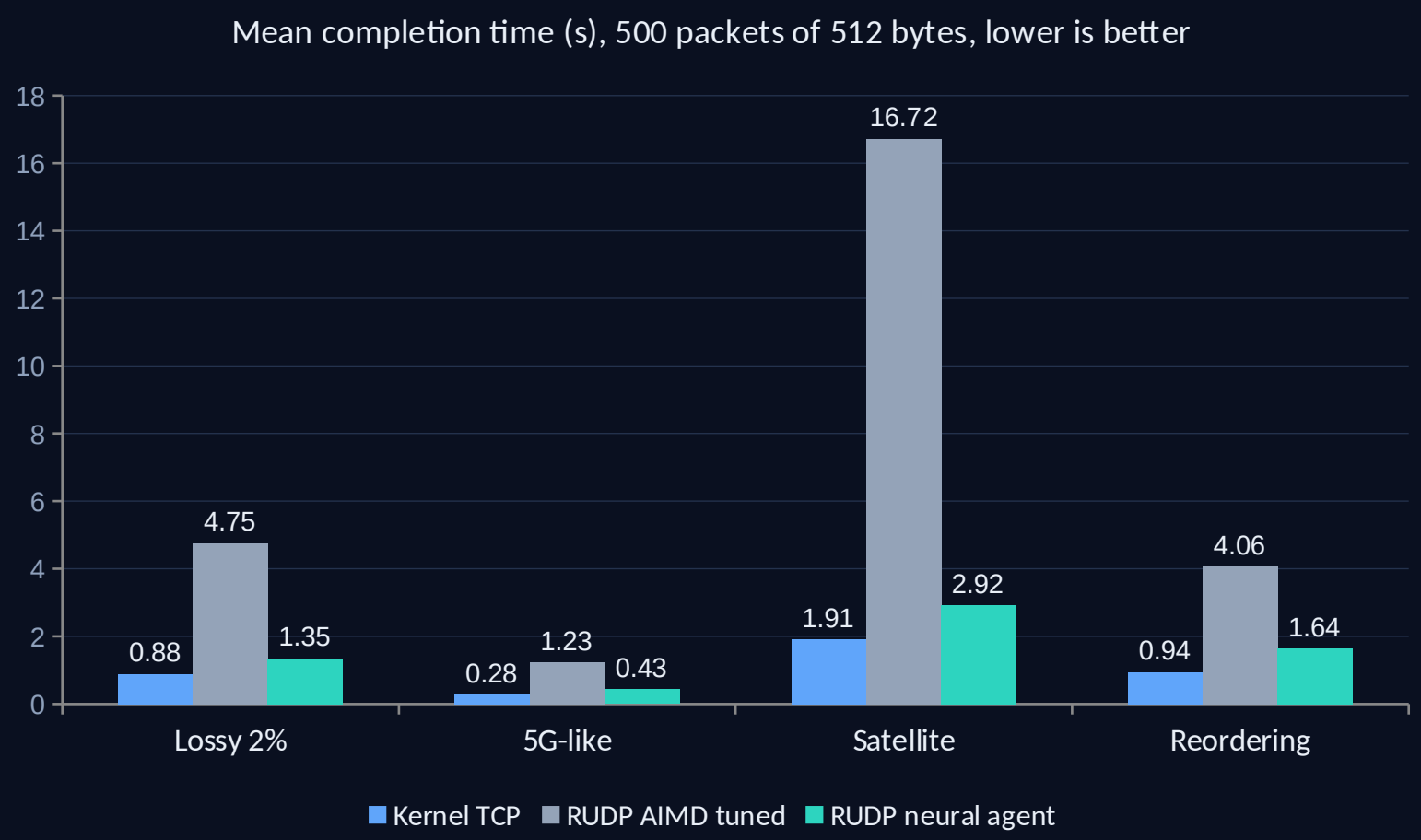
# Why it wins: it rides a higher window and backs off only on loss




- Same lossy path, same seed, same transport
- AIMD (grey) saws between 5 and 15 packets and needs 9 s
- The agent (green) holds a much larger window, backs off on loss, and finishes in 4 s

RESULTS

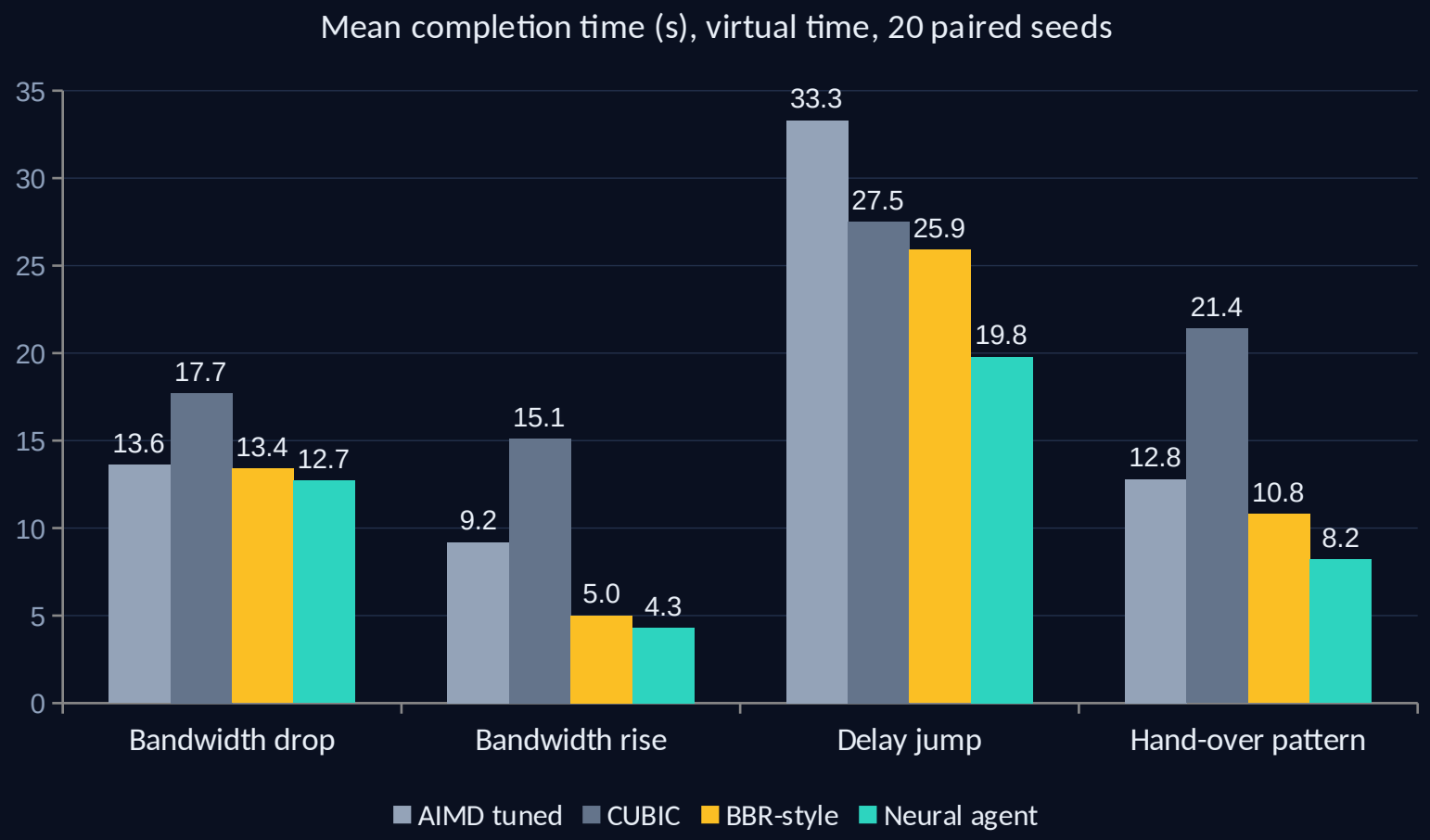
# On the real Linux kernel: 2.5x to 6x faster than AIMD, within 1.5x to 1.7x of TCP




- Linux tc/netem on loopback, 5 runs per cell (satellite 3)
- Neural agent beats tuned AIMD by 65% to 83%; significant on three paths, reordering not
- Kernel TCP is faster than any user-space RUDP policy, as expected
- Next to a kernel TCP flow our agent takes 24% to 34% of the link

RESULTS

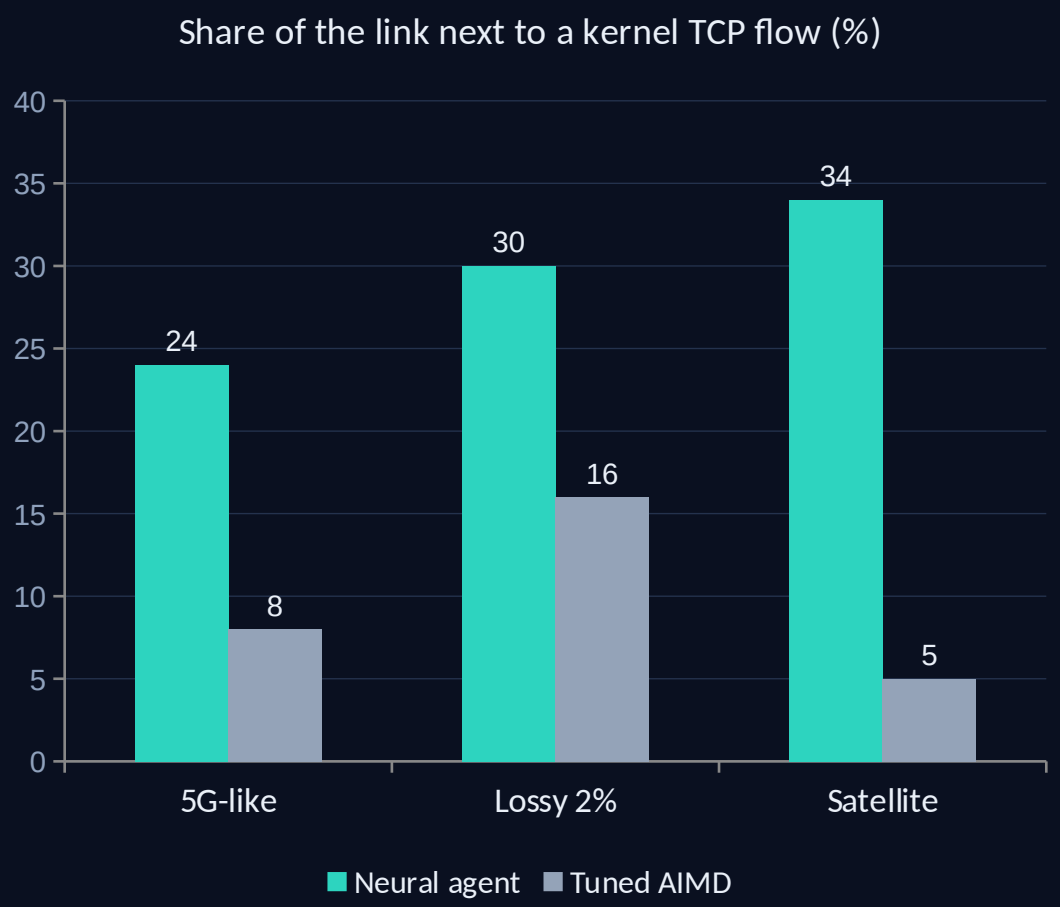
# When the path changes mid-transfer, the neural agent is fastest every time




- 4 of 4 changing-path cases: neural fastest
- Cost: 1% to 6% retransmissions
- The BBR-style baseline beats tuned AIMD on 4 of 5 named paths, so this is a strong opponent
- Simplified BBR in virtual time; not the Linux BBR

RESULTS

# Fair between neural flows, and it does not starve kernel TCP

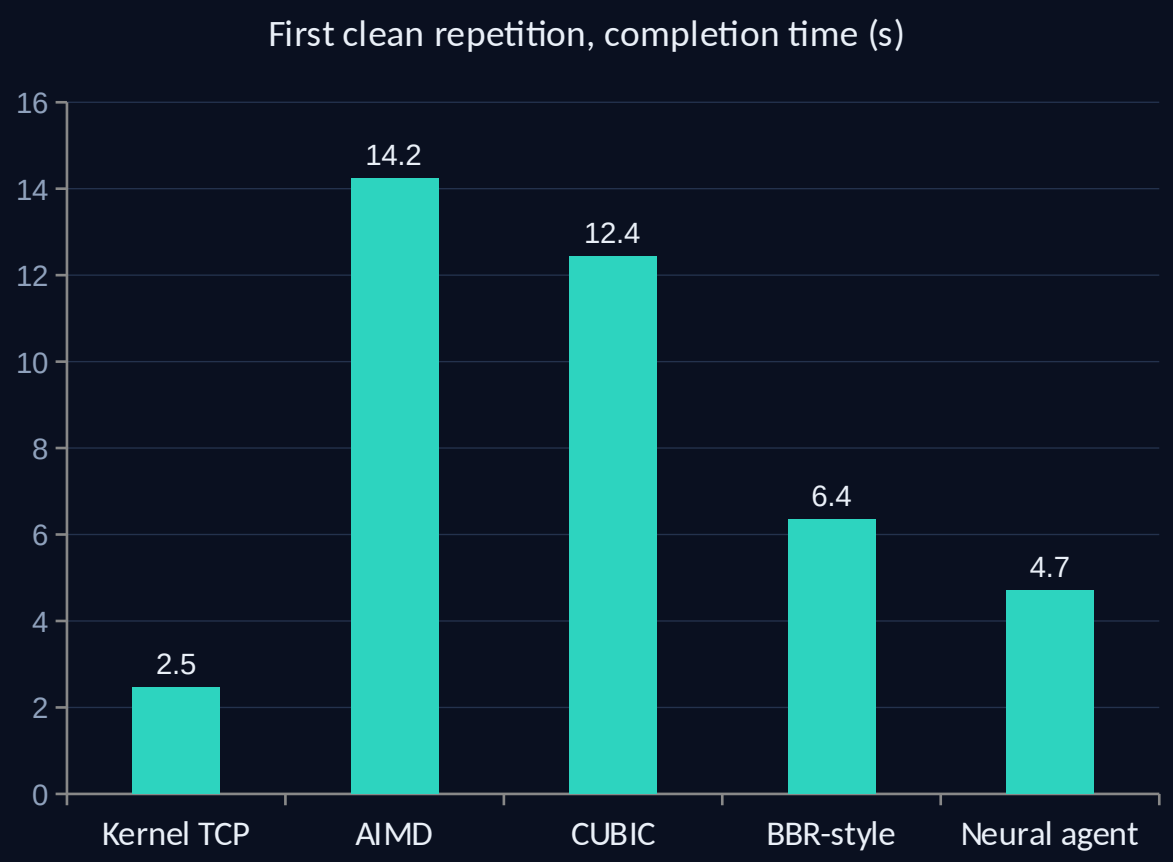



| Two identical flows | Extra RTT | Share short-RTT | Jain |
|---|---|---|---|
| AIMD / AIMD | 150 ms | 88% | 0.64 |
| Neural / neural | 150 ms | 71% | 0.84 |

- TCP keeps the rest of the link in every case
- Neural flows are fairer than AIMD under unequal RTTs; the short-RTT flow still wins more
- Coexistence: 3 runs per cell on one laptop, so shares are good to about ten points

RESULTS

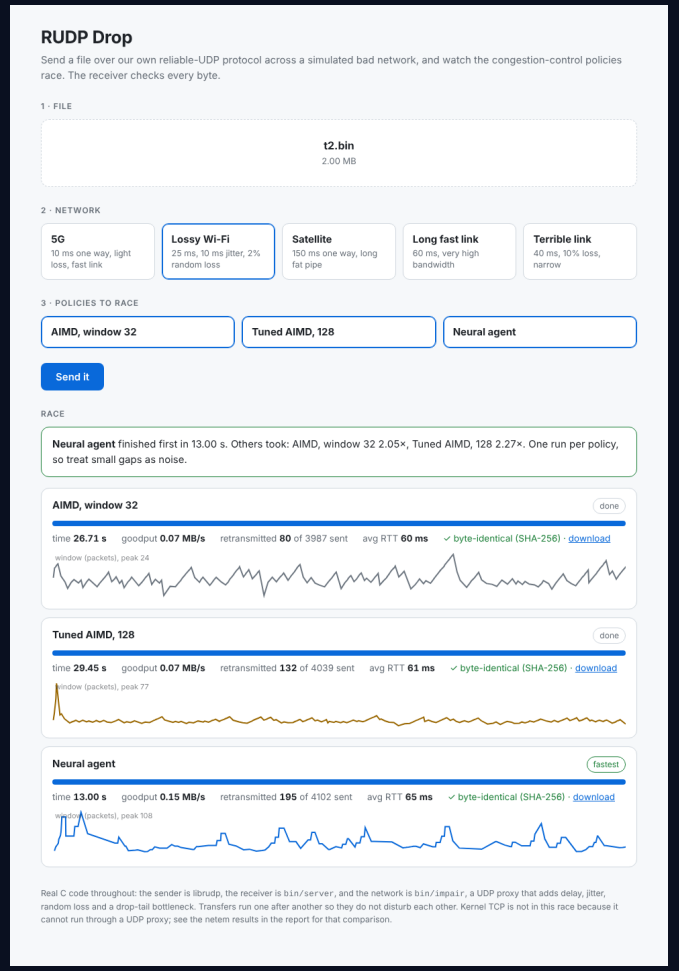
# It works between two real machines over the internet




- Two laptops on different networks, Tailscale relay, about 79 ms round trip; 2 MB per transfer, 5 repetitions
- Every completed RUDP transfer delivered 4,000 of 4,000 packets
- Rep 1: neural 4.7 s, BBR-style 6.4 s, AIMD 14.2 s, kernel TCP 2.5 s
- The relayed path drifted (TCP alone took 2 s to 51 s) and 4 of 20 runs timed out, so no ranking is claimed from reps 3 to 5

DEMO

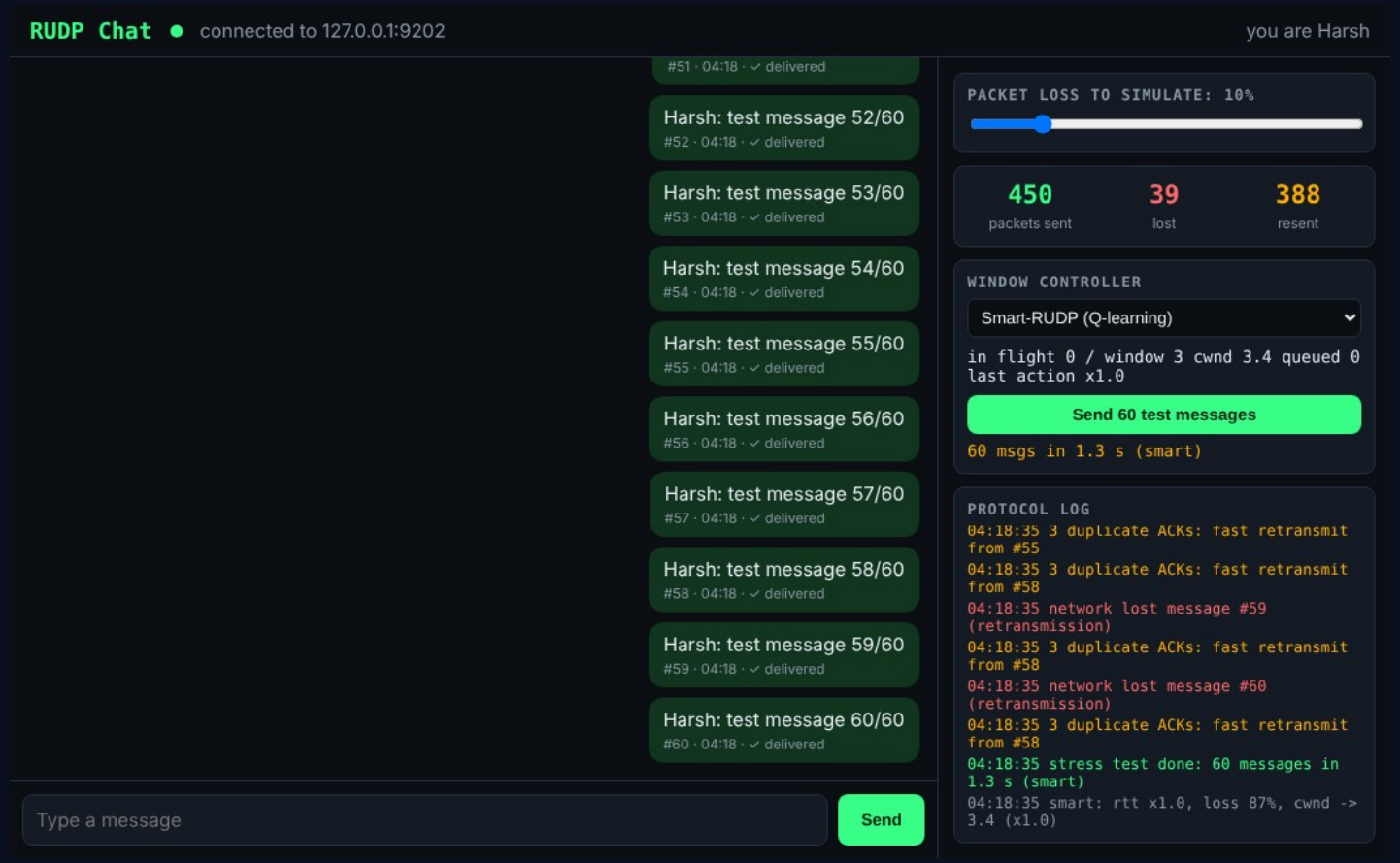
# RUDP Drop: race the policies on any file




- Drop in any file (up to 12 MB), pick a network, race AIMD 32, tuned AIMD and the neural agent
- Example, 2 MB on lossy Wi-Fi: neural 13.0 s, AIMD 26.7 s, tuned AIMD 29.5 s (one run each)
- Each received copy is checked byte for byte (SHA-256) and can be downloaded
- Real C stack: librudp sender, bin/server and the bin/impair proxy; make drop starts it

DEMO

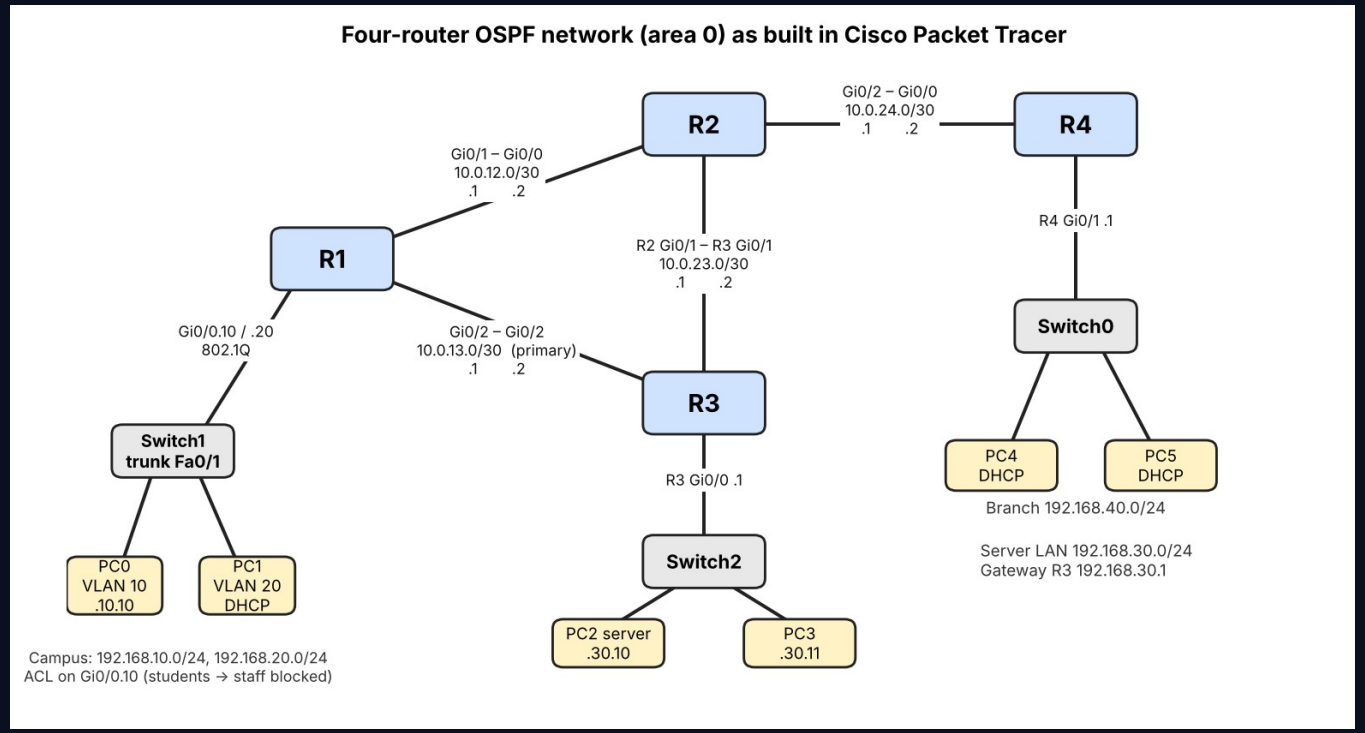
# RUDP Chat: two people, two machines, over the internet




- Every message is a RUDP segment; ACKs and retransmissions are visible in the live counters
- Two machines linked over Tailscale; works across the internet
- Window controller switch: fixed window, AIMD or Smart-RUDP
- Loss slider shows the protocol recovering in real time

DEMO

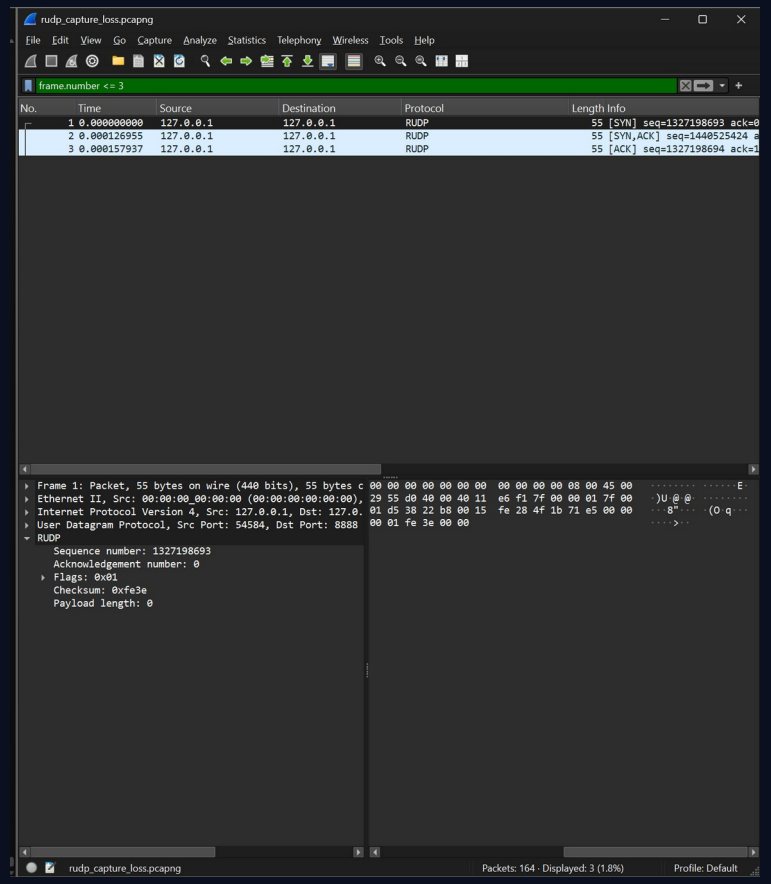
# Network design in Packet Tracer: OSPF, VLANs, ACL and failover

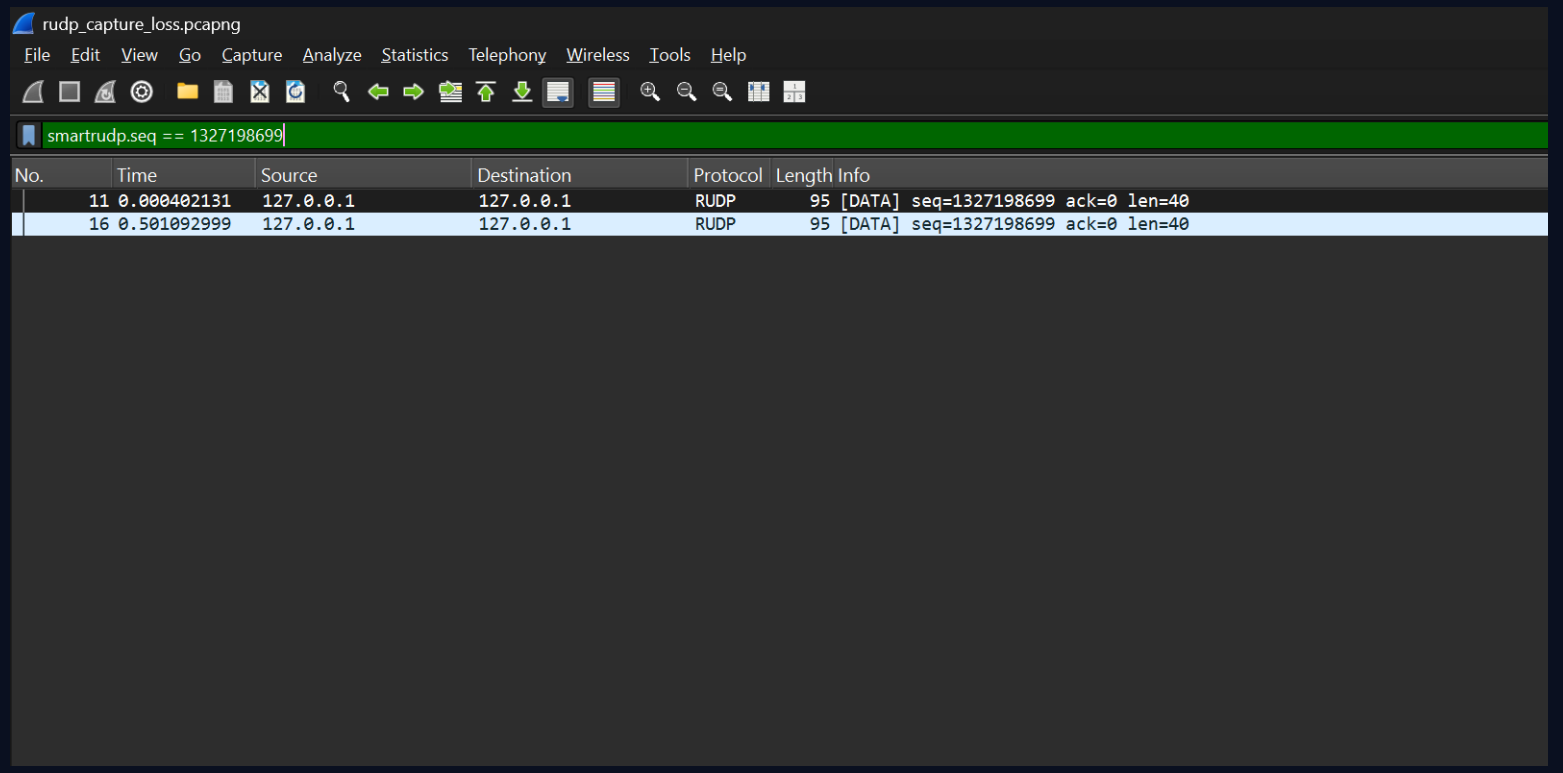



- 4 routers, OSPF area 0; 2 VLANs with router-on-a-stick and DHCP
- Student PCs are blocked from staff VLAN 20 by an ACL
- Failover: shutting R1 Gi0/2 reroutes through R2; 70 of 71 pings still succeed
- Packet Tracer cannot run C programs, so protocol and network are validated separately

DEMO

# Wireshark: a custom dissector decodes RUDP

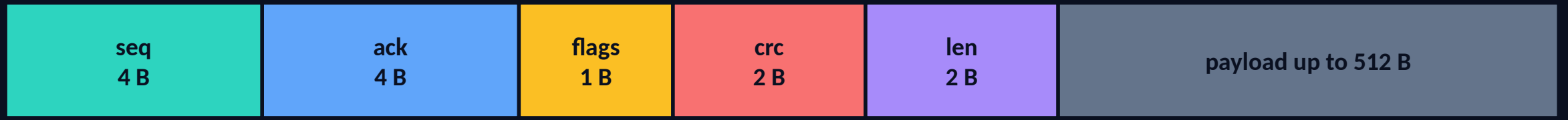



Dissector network/rudp.lua decodes seq, ack, flags and CRC. Left: the 3-way handshake. Right: filtering one sequence number shows its retransmissions.

HOW IT WORKS

# RUDP: a 13-byte header, a three-way handshake and a CRC on every packet




| | | |
|---|---|---|
| **SYN** | Client to server | Request a connection |
| **SYN-ACK** | Server to client | Server agrees |
| **ACK** | Client to server | Established; data flows |
| **DATA / ACK** | Both ways | Window of packets in flight |
| **FIN** | Either | Close the connection |

Packets are numbered, acknowledged and checksummed; a corrupt packet is dropped and recovered by retransmission.

HOW IT WORKS

# Selective Repeat: resend only what is missing

- Receiver buffers out-of-order packets (256-packet window) and reports them in a 32-byte bitmap in every ACK
- Sender keeps one timer per packet; adaptive timeout with a floor of 1.3 x smoothed RTT
- About 10x fewer retransmissions than Go-Back-N
- Web chat, 200 messages: 6.4 s against 11.9 s with AIMD
- Go-Back-N kept for comparison (--gbn)

| C transport (s) | AIMD 32 | Neural | Gain |
|---|---|---|---|
| 5G-like | 1.27 | 1.16 | 9% |
| Lossy 2% | 7.99 | 4.29 | 46% |
| Satellite-like | 25.7 | 12.9 | 50% |
| Hostile 10% loss | 7.3 | 8.1 | tie |

10 runs per cell, 95% intervals; all gains significant except the hostile row.

HOW IT WORKS

# Five congestion controllers behind one interface

### AIMD

Add 1 per RTT, halve on loss. The TCP Reno idea.

### CUBIC

Window grows as a cubic curve after loss.

### BBR-style

Estimates bandwidth and RTT; paces at the estimate.

### Q-table

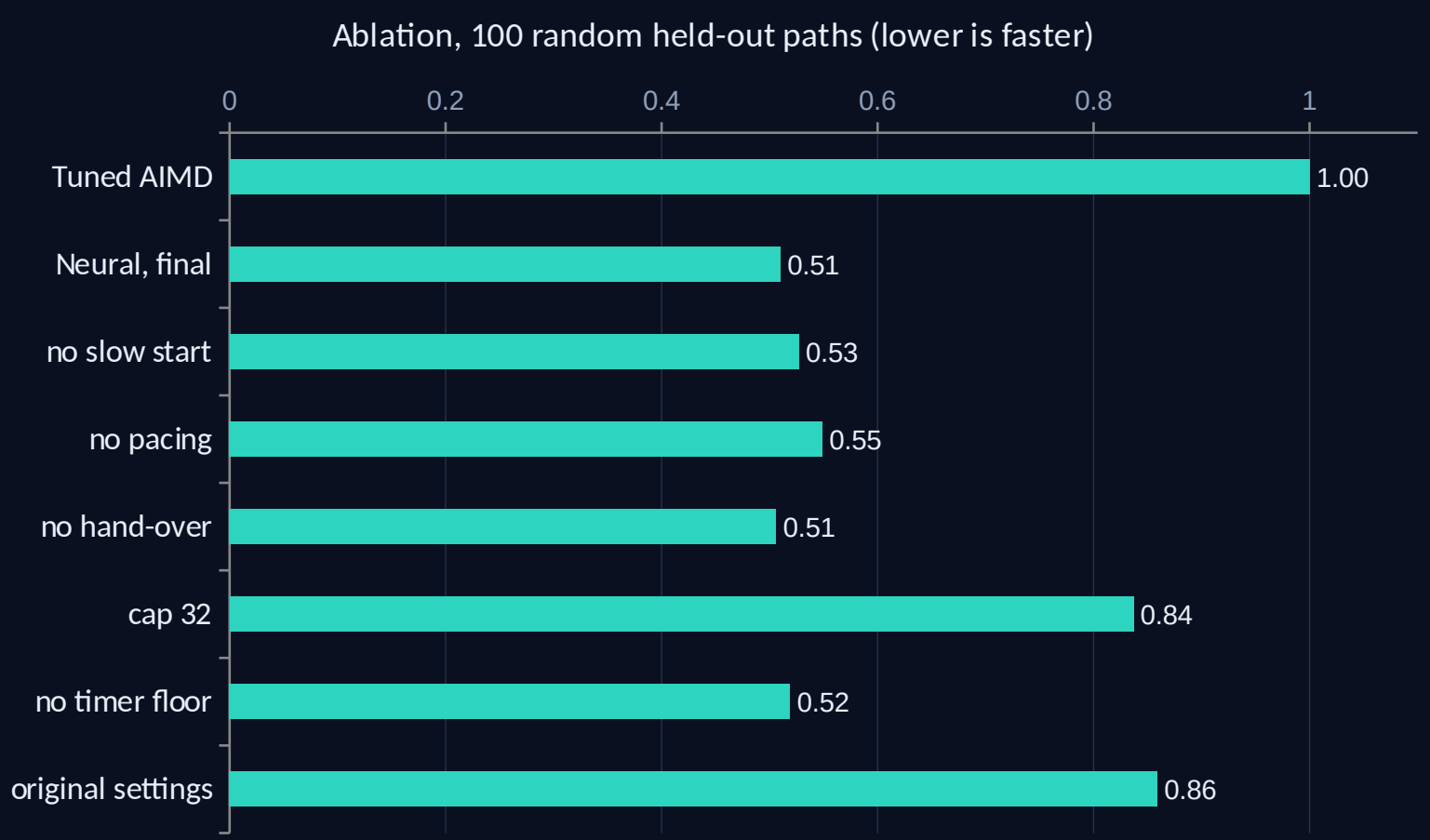
Small learned table of window actions.

### Neural agent

5-32-32-5 network, 1,413 weights, picks the window multiplier.

- Inputs: four measured values (RTT growth, loss, throughput share, window) plus the last action
- Five actions: multiply the window by 0.5, 0.85, 1.0, 1.15 or 1.5
- Window cap 128, slow start to 64, pacing at 2 x window / RTT
- Safety net: AIMD takes over while an interval loses 10% or more

# What matters: window cap 128, pacing, slow start and the timer floor




- 100 random paths: neural 0.53x the time of tuned AIMD, faster on 89, never more than 20% slower
- Removing the cap 128 or using the original settings loses most of the gain
- Evolution-strategy retraining in C added only 2%: not adopted

TESTING

# How we tested: correctness first, then four steps toward the real world

### 1 Emulation in the client

Fast, repeatable; also virtual time (1000x faster, matches the real clock in 15 of 15 cells)

### 2 Impairment proxy

Separate process on real UDP sockets adds delay, jitter, loss and a bottleneck

### 3 Linux kernel netem

Real kernel queues and loss on loopback, against kernel TCP

### 4 Two real machines

Laptops on different networks over the internet

- Correctness: unit, end-to-end, handshake, file-integrity, chat and simulator tests (make test); packet parser fuzzed under AddressSanitizer and UBSan (make fuzz)
- Statistics: paired seeds, 95% intervals, tuning seeds kept apart from test seeds
- Every result has the command that produced it (docs/RUNNING_ALL_TESTS.md)

ISSUES FOUND

# What we found along the way

- The first learned controller (a Q-table) was not faster than AIMD
- At 10% random loss the neural agent was 7x slower than AIMD (85 s against 11 s)
- Real sockets exposed a lost-handshake hang, spurious timeouts (262 retransmissions for about 3 losses) and a weak checksum
- Our early baselines were weak: untuned AIMD (window cap 32) made first wins look bigger
- Our own kernel tests were wrong four times: netem limit, untuned baseline, unequal payloads, synthetic 40-byte packets
- The first BBR-style baseline stalled at 10% loss and took 163 s after a delay jump
- The agent is a pushier neighbour than AIMD or CUBIC, and results vary with the training seed

WHAT WE CHANGED

# Every issue led to a fix

| Before | After |
| --- | --- |
| Go-Back-N with a fixed 500 ms timer | Selective Repeat: bitmap ACKs, per-packet timers, adaptive timer with a 1.3 x srtt floor; about 10x fewer retransmissions |
| Byte-sum checksum | CRC-16: 0 of 333k corrupted packets accepted |
| Server ignored data after a lost final handshake ACK | Handled as in TCP, with a regression test |
| Q-table agent, window cap 32 | 5-32-32-5 neural network, cap 128, slow start, pacing |
| Neural agent 7x slower than AIMD at 10% loss | Hand-over to AIMD while loss is high: 13.9 s instead of 84.9 s |
| Untuned AIMD and CUBIC baselines | Tuned baselines, so every win is against a fair opponent |
| Faulty kernel and BBR tests | Netem limit, equal payloads and real 512-byte packets; kernel runs repeated; BBR stall and slow recovery fixed |

LIMITS

# What we state openly

- Kernel TCP is still 1.5x to 1.7x faster than the neural agent on every kernel path
- Against tuned AIMD the agent takes about 70% of a shared link; the optional polite mode fixes that but costs speed
- Level with CUBIC under bursty cross traffic; no gain at 10% random loss; the BBR-style baseline is faster on the 2% lossy path
- Not formally verified: certified at +-0.001 for only 43% of decisions; trained in a simulator, never on real traces
- Two-machine run: one drifting relay path, 4 of 20 runs timed out, so no ranking is claimed
- Coexistence with kernel TCP: 3 runs per cell

WRAP-UP

# What we learned and what comes next

## Learned

Reliability is a set of simple rules working together.

Congestion control matters as much as retransmission.

A tuned baseline is the only fair opponent.

Test tools fail in surprising ways, so check your own numbers.

## Next

Controlled multi-machine tests on a direct network

Train with a competing flow so fairness is learned

Train on real traces with cross traffic

Close the gap to kernel TCP

# Thank you

github.com/Capsturrrr/rudp

Harshavardhan Mallela  Rohit Arya  Shloak Sinha

# Neural agent in the C transport

| 1,200 packets, C transport | AIMD | Table (rl2) | Neural |
|---|---|---|---|
| 5G-like | 1.38 s | 2.64 s | 1.18 s |
| Lossy wireless | 9.13 s | 7.54 s | 6.63 s |
| Satellite-like | 17.9 s | 38.1 s | 13.1 s |

15 runs per cell (satellite: 8), 95% intervals. Equal-cap check (AIMD capped at 32, 10 runs): neural still wins by 19%, 29%, 28%, all significant (eval_ci4.sh).

- Neural agent beats AIMD on all three paths: 15%, 27%, 24% faster (95% intervals exclude zero)
- Table agent does not generalise: slower on 5G-like and satellite-like
- Selective ACK in C: about 10x fewer retransmissions, no significant time change here
- Caveats: trained in a simulator; emulated paths (not real networks); agent retransmits more on random loss

# Safety net: hand over to AIMD

| C transport, seconds | AIMD 32 | Neural | Hybrid |
|---|---|---|---|
| 5G-like | 1.48 | 1.17 | 1.23 |
| Lossy 2% | 9.04 | 7.53 | 7.57 |
| Satellite-like | 19.7 | 13.4 | 13.7 |
| Hostile, 10% loss | 11.1 | 84.9 | 13.9 |

10 runs per cell (hostile: 5 runs, 300 packets), 95% intervals. Flag: --hybrid 0.10. Code: eval_ci6.sh.

- Problem found: at 10% loss the neural agent is 7x slower than AIMD
- Fix: while a decision interval loses 10% or more, AIMD drives the window; after 3 clean intervals the agent resumes
- Keeps the wins on the three benign paths, and the hostile path is no longer significantly different from AIMD
- A timeout cut (x0.25) was tried and dropped: it helped the hostile path but hurt the 2% path (24 s vs 9 s)
- Honest summary: the agent helps on clean-to-moderate paths; it needs a classical safety net elsewhere

# More evidence: baselines, SACK, reordering

| 48 paths, original simulator | Mean | Median |
|---|---|---|
| Plain UDP | 68% delivered | - |
| TCP model (selective ACK) | 4.56 s | 1.48 s |
| RUDP fixed window | 3.88 s | 2.00 s |
| RUDP AIMD | 7.16 s | 2.72 s |
| Smart-RUDP, table | 3.73 s | 1.99 s |
| Smart-RUDP, deep | 3.20 s | 1.79 s |

- Selective ACK (simulated): AIMD mean 7.16 s to 4.90 s, median 2.72 s to 1.44 s; faster on 34 of 48 paths. Go-Back-N is the main weakness.
- Reordering (20 ms jitter): AIMD 0.82 s to 6.79 s; deep agent 0.93 s; SACK removes most of the damage.
- 20-run intervals for the C experiments: Smart slower on 5G-like (+0.92 s) and satellite-like (+6.3 s), tie on lossy.

Caveat: this simulator allowed a dup-ACK retransmission storm. With the realistic rule, stress means are AIMD 7.75 s, neural 7.27 s: about level. The TCP row is a model, not kernel TCP.

# Fairness, ablations, chat fixes

| Two flows, one bottleneck | Share of A | Jain |
|---|---|---|
| AIMD vs AIMD (long path) | 51% | 0.999 |
| Deep vs deep (long path) | 59% | 0.969 |
| Deep vs AIMD (no-loss path) | 44% | 0.987 |
| Deep vs AIMD (long path) | 74% | 0.819 |
| Fixed 8 vs AIMD (long path) | 72% | 0.843 |

Simulator, 20 seeds; ablation on 48 stress paths. Code: web/fairness_test.py, web/ablate.py.

- Fair on short paths; takes 74% against AIMD on the 120 ms path, like a fixed window
- Ablation: the no-shrink mask is what matters (6.54 s without it, about 3.3 s with it; AIMD 7.16 s)
- Other parts (delay penalty, last action, throughput input) showed no measurable value
- Same recipe gave 3.2 to 7.4 s across seeds: seed sensitivity is a real weakness
- Found and fixed a dup-ACK retransmission storm in the chat gateway; added a live dashboard and an adaptive RTO

# Fairness and file transfer

| Two flows, real sockets | Share A | Share B | Jain |
|---|---|---|---|
| AIMD vs AIMD | 49% | 51% | 0.99 |
| Neural vs neural | 50% | 50% | 0.97 |
| Neural vs AIMD tuned | 68% | 32% | 0.88 |
| Neural vs AIMD cap 32 | 76% | 24% | 0.78 |
| CUBIC vs AIMD tuned | 61% | 39% | 0.93 |

| File transfer | AIMD cap 32 | AIMD tuned | Neural |
|---|---|---|---|
| 2 MB, lossy 2% | 26.6 s | 24.2 s | 11.9 s |
| 4 MB, fast path | 34.6 s | 21.0 s | 8.9 s |

- The agent is a more aggressive neighbour than AIMD or CUBIC: no starvation, but not polite. Not trained for fairness; TCP coexistence untested
- smart_client --file sends a real file; the received file is byte-identical in every run (FNV-1a hash)
- Terminal chat and web gateway now use the same 256-packet window and 32-byte bitmap as the C transport; interop test at 20% loss
- CI, fuzzing, MIT licence, architecture doc

# Closing gaps: fairness and cross traffic

| Neural vs tuned AIMD | Agent share | Jain |
|---|---|---|
| Default | 70% | 0.86 |
| --polite 1.2 | 49% | 0.95 |
| --polite 1.3 | 52% | 0.94 |
| --polite 1.5 | 59% | 0.90 |

| Bursty cross traffic | Time | vs AIMD |
|---|---|---|
| AIMD tuned | 5.73 s | 1.00 |
| CUBIC tuned | 4.55 s | 0.79 |
| Neural agent | 4.87 s | 0.85 |
| Neural, polite 1.3 | 4.92 s | 0.86 |

- Polite mode: back off when a standing queue builds (jitter-aware). Share 70% to 52%; costs speed (0.53x to 0.58x of AIMD time on 100 paths, worst path 2.95x slower), so it is off by default
- Bursty cross traffic (new proxy option, 8 runs): neural is 15% faster than tuned AIMD but level with CUBIC; three tweaks to fix its extra retransmissions did not help
- Server now handles up to 16 clients at once; a test sends two files simultaneously and checks both byte-for-byte

# Real sockets found what the emulator could not

### Lost handshake ACK

Server ignored all data after a lost final ACK: transfers hung (2 of 4 runs at 10% loss). Fixed as in TCP; regression test added.

### Spurious timeouts

Timer too tight for window bursts: 262 retransmissions for ~3 losses. Floor of 1.3 x srtt. It also made AIMD 41% faster, so the old margins were too big.

### Weak checksum

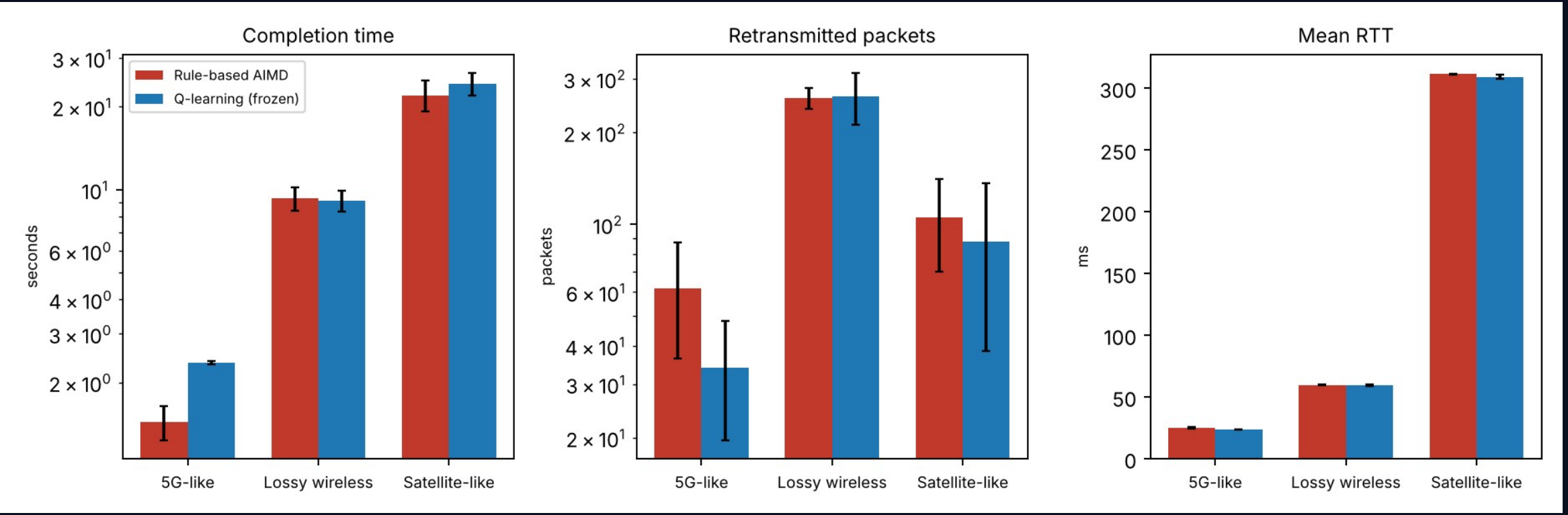
Byte-sum checksum let 2% of corrupted packets through (fuzzer). Now CRC-16: 0 of 333k accepted, sanitizers clean.

Independent check on real UDP sockets through an impairment proxy (src/impair.c): neural agent -17% / -53% / -45% / -39% vs tuned AIMD on 5G / lossy / satellite / fast; hostile path a tie. eval_proxy.sh

# Problems we hit and fixed

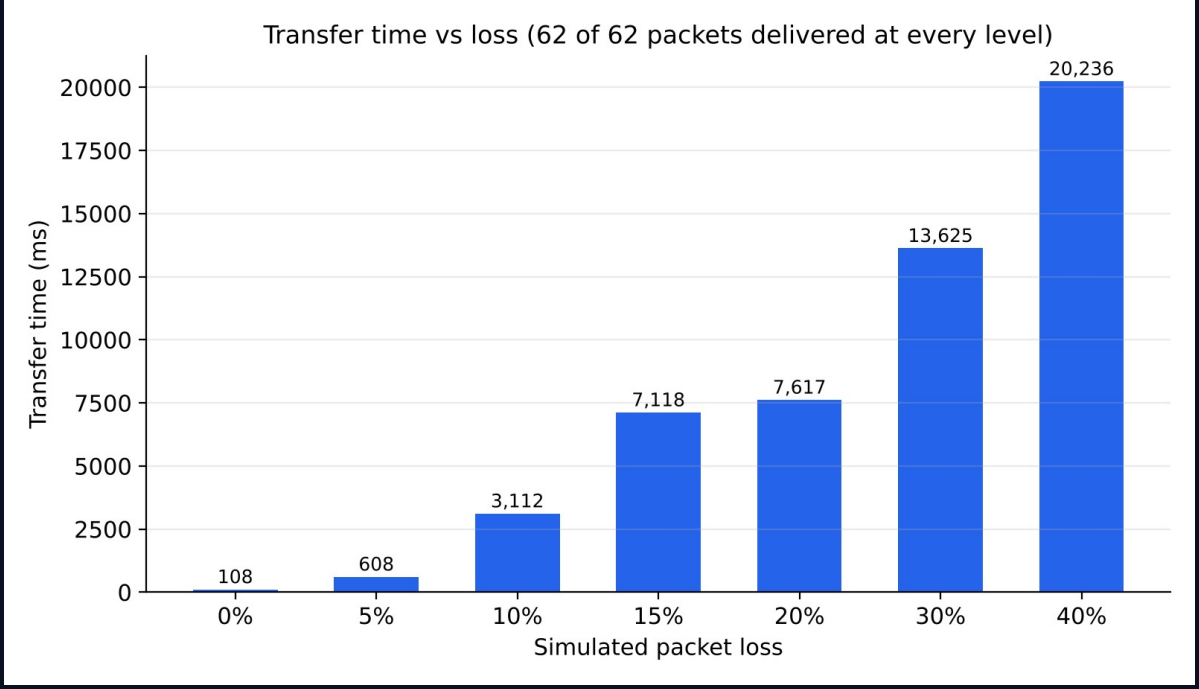
| Problem | Fix |
| --- | --- |
| Server stuck after killed client | Accept SYN in any state (was a 15.5 s hang) |
| Leftover netem rule | Check tc qdisc show dev lo, delete before each test |
| Windows Wireshark saw nothing | Capture with tshark inside WSL, then copy |
| Shell portability in benchmark | Replaced date/grep tricks with awk and sed |

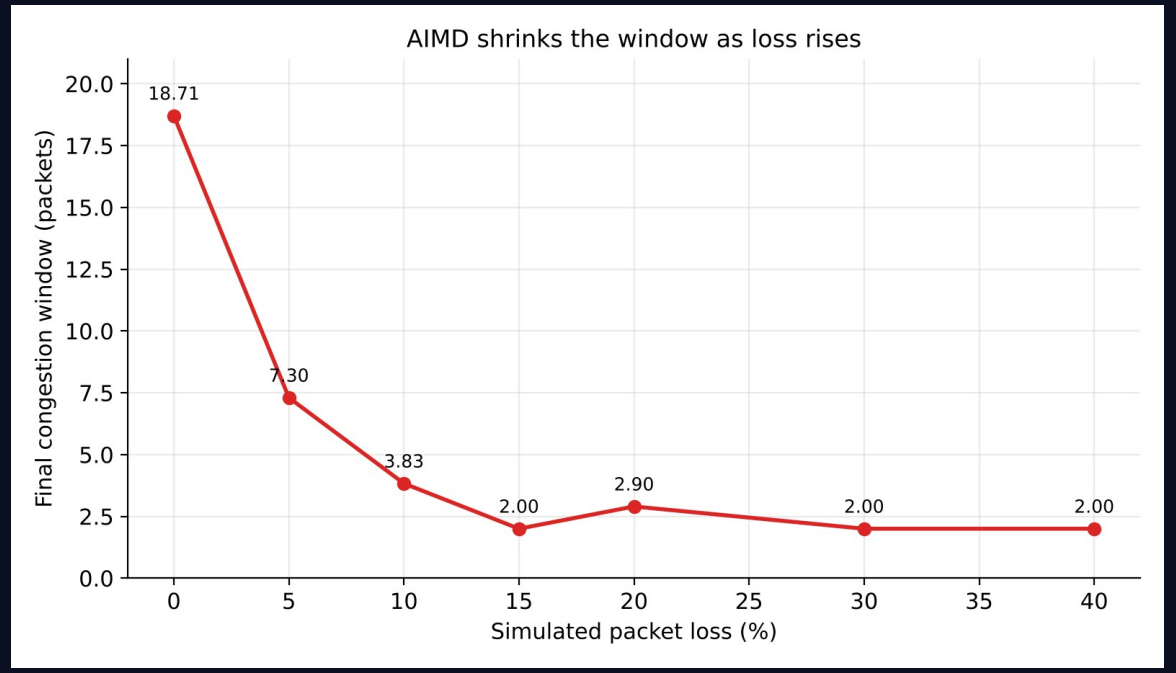
# Our first learned agent: an honest result




- 6 evaluation runs per scenario after 40 training episodes
- Not faster: +63% time on 5G-like, +10% on satellite, tie on lossy
- Fewer retransmissions on 5G-like and satellite, but within run-to-run noise
- Earlier training rounds were much worse; reported run is the final configuration

# Benchmark results




**62 of 62 packets delivered at every loss level, from 0% to 40%.**

# Literature behind the design

- Jacobson & Karels (1988): congestion avoidance and control, the origin of AIMD
- Chiu & Jain (1989): why additive increase, multiplicative decrease converges fairly
- RFC 5681 (2009): TCP congestion control standard
- Ha, Rhee, Xu (2008): CUBIC, our future-work direction
- Langley et al. (2017) and RFC 9000: QUIC, reliable transport over UDP in production
- Midhula & Arun Raj Kumar (2024) and Cao et al. (2023, DRLFcc): deep reinforcement learning for congestion control, the idea behind Smart-RUDP

**Key researcher: Van Jacobson**

# Smart-RUDP: a Q-learning window controller

### State (80)

RTT ratio x loss fraction x window size, sampled once per RTT.

### Action (5)

Multiply cwnd by 0.5, 0.85, 1.0, 1.15 or 1.5.

### Reward

Throughput gained, minus delay growth, minus retransmissions.

Same Go-Back-N reliability as the baseline; only the window policy changes. Paths emulated in-program: 5G-like, lossy wireless, satellite-like.

# The AI agent in our code

**src/rl_cc.h (agent, simplified)**

```
// state: 4 x 4 x 5 = 80 buckets
s = (rtt_bucket*4 + loss_bucket)*5 + cwnd_bucket;

// choose: explore with prob. eps, else best
int rl_choose(a, s) {
  if (rand01() < a->eps) return rand() % 5;
  return argmax(a->q[s]);        // 5 actions
}

// learn: the Q-learning update
void rl_update(a, s, act, r, s2) {
  target = r + a->gamma * max(a->q[s2]);
  a->q[s][act] += a->alpha * (target - a->q[s][act]);
}
```

**src/smart_client.c (decision loop, simplified)**

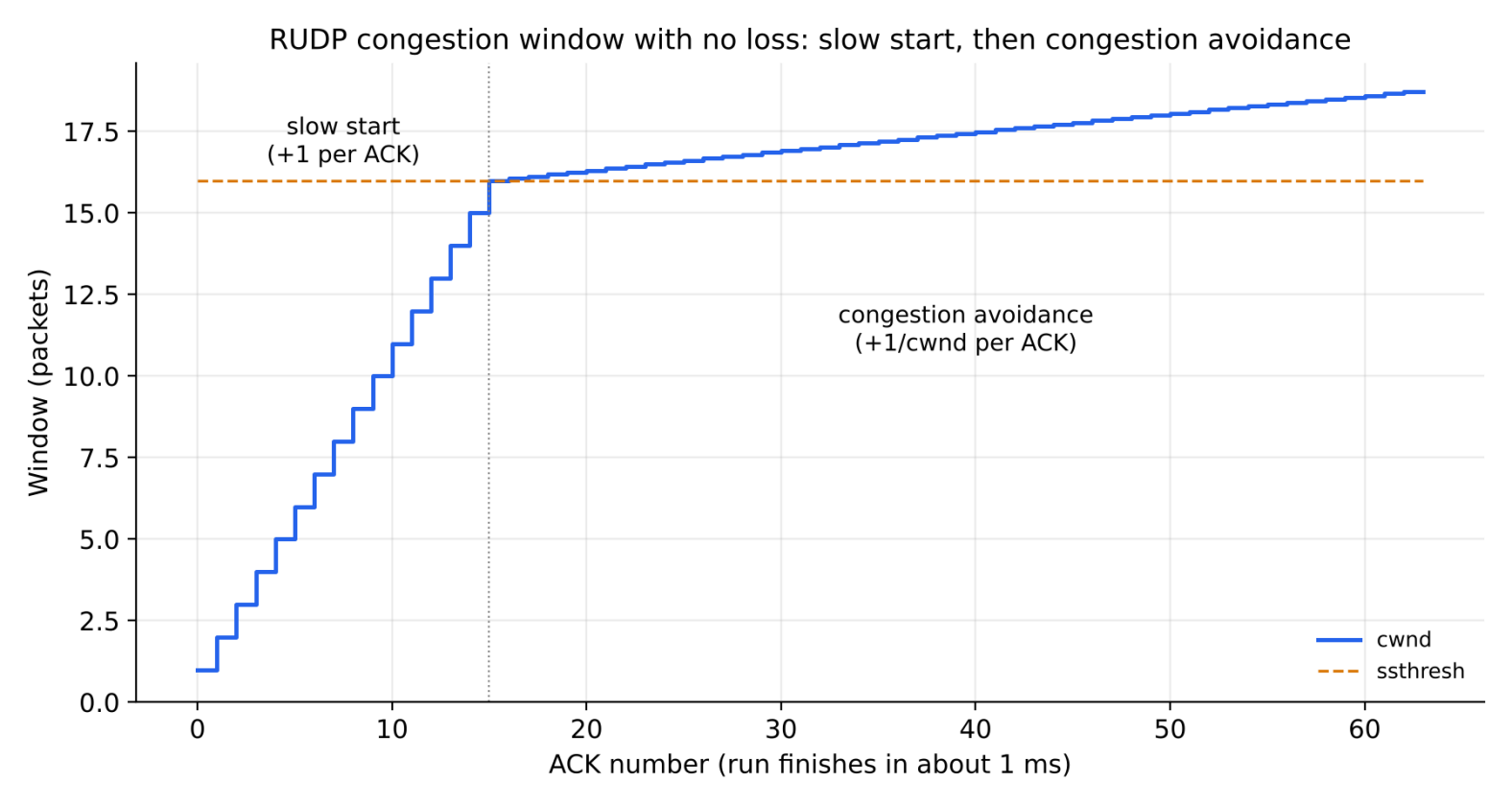
```
// once per RTT, in smart_client.c
ratio = rtt / base_rtt;   lossf = losses / sent;

reward = sqrt(throughput_norm)
       - 0.5 * max(0, ratio - 1)   // delay growth
       - 4.0 * lossf;              // loss

rl_update(&ag, prev_s, prev_a, reward, s2);
act   = rl_choose(&ag, s2);
cwnd *= {0.5, 0.85, 1.0, 1.15, 1.5}[act];
```

Tabular Q-learning: alpha 0.15, gamma 0.9. Chat version: web/cc.py, trained in web/train.py (results/q_chat.txt). Deep version: web/dqn.py, a small neural network (5-32-32-5) trained by DQN in web/train_deep.py.

# Appendix: slow start and congestion avoidance




- cwnd starts at 1
- +1 per ACK until ssthresh = 16
- Then +1/cwnd per ACK
- 62 packets, final cwnd 18.71